NOTES

HOW AN AGENTIC APP WORKS, IN PLAIN WORDS

Memorandum on Agentic AI Applications · 21 Oct 2026

Contents

I. The big picture
II. The one loop every agent uses
III. The full flow, step by step
IV. How the project is organised
V. All the types of agent apps, with examples
VI. Five rules every real agentic app follows
VII. In one line
VIII. Reading list

Every red underlined title is a clickable link to the paper or guide that explains that part in full.

I. THE BIG PICTURE

An agentic app has three parts working together:

1. The app. This is what the user sees and touches: website, phone app, chat window, or bot.
2. The server. It sits in the middle. It checks who the user is, keeps the secret keys safe, and calls the agent.
3. The agent. This is the "brain". It is an AI model plus instructions plus tools, and it runs in a loop until the job is done.

The user never talks to the AI model directly. They talk to the app, the app talks to the server, and the server runs the agent.

> READ THE PAPER
> ▸ Building effective agents (Anthropic, 2024): the clearest guide to what an agent is and the common app patterns.
> ▸ LLM Powered Autonomous Agents (Lilian Weng, 2023): model + planning + memory + tools, explained with diagrams.

II. THE ONE LOOP EVERY AGENT USES

Each time the agent gets a task, it repeats the same cycle:

1. Think. It reads the task and decides: "Can I answer now, or do I need more information?"
2. Use a tool. If it needs information, it picks a tool, such as "look up order", "search docs" or "run tests".
3. Read the result. The tool's answer goes back to the agent.
4. Repeat or finish. If it has enough, it writes the final answer. If not, it goes back to step 1.

A safety limit stops the loop after a set number of rounds, so it can never run forever.

Example: "Where is my order 123?"

- Think: "I don't know this. I need the order data."
- Tool: looks up order 123
- Result: "Shipped, arrives 22 Oct."
- Think: "Now I can answer."
- Final answer: "Your order has shipped and will arrive on 22 October."

> READ THE PAPER
> ▸ ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2022): the paper behind the think, use a tool, read the result loop.
> ▸ Toolformer: Language Models Can Teach Themselves to Use Tools (Schick et al., 2023): how a model learns when and how to call a tool.

III. THE FULL FLOW, STEP BY STEP

Step 1: The user asks. The user types or speaks a request in the app.

Step 2: The app sends it to the server. The app does not think. It only passes the message on.

Step 3: The server checks the user. It confirms the user is logged in and notes who they are. This matters because the agent must only see this user's data.

Step 4: The server starts the agent. It gives the agent three things:

- the user's message
- the instructions (who it is, what it may and may not do, how to reply)
- the list of tools it may use for this user

Step 5: The agent runs the loop. It thinks, uses a tool, and reads the result, as many times as needed.

Step 6: The tools do the real work. Tools are the agent's hands. They read the database, call other services, and send messages. The agent only decides which

tool to use and what to ask it.

Step 7: The answer streams back. The words come back through the server to the app and appear on screen word by word, so the user isn't left waiting.

Step 8 (optional): Long jobs. If the task takes minutes or hours (a report, 100 emails), the server puts it in a queue. A background worker runs the agent, and the user gets a notice when it is done.

READ THE GUIDE

- Tool use with Claude (Anthropic docs): how the server hands tools to the model and runs them.
- Streaming messages (Anthropic docs): how the answer comes back word by word.
- Durable execution (LangGraph docs): long jobs that survive restarts and run in the background.

IV. HOW THE PROJECT IS ORGANISED

FOLDER	WHAT LIVES THERE	EXAMPLE
App / screens	What the user sees	chat page, dashboard, settings
Server routes	The middle layer	"receive chat message", "check login"
Agent folder	The brain	instructions, the list of tools, model choice
Tools folder	The agent's hands	"get order", "search docs", "book slot"
Database / shared	Data both sides use	users, orders, notes
Worker	Long background jobs	reports, scheduled tasks
Tests / evals	Checks the agent answers well	a list of sample questions and expected answers

Keeping the agent in its own folder means you can change the AI model or improve the instructions without touching the rest of the app.

READ THE GUIDE

- 12-Factor Agents (HumanLayer): rules for building agents that are real, testable software.
- A Practical Guide to Building Agents (OpenAI, PDF): model, tools, instructions, and guardrails as separate parts.
- Writing effective tools for agents (Anthropic): how to design the tools folder and test it with evals.

V. ALL THE TYPES OF AGENT APPS, WITH EXAMPLES

1. Single agent with tools (the basic type)

One agent, a few tools, and one loop.

- Flow: User asks, the agent looks something up, then answers.
- Examples:
  - FAQ bot on a website that searches the help pages
  - Order status bot that looks up the order
  - Repo Q&A bot: "Where is login handled?" It searches the code and answers.
  - Log reader that reads server logs and explains the errors

> READ THE PAPER
> ▸ ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2022): the paper behind the think, use a tool, read the result loop.
> ▸ MRKL Systems (Karpas et al., 2022): early design of one model routing work to tools.

2. Router + specialist agents (handoff / multi-agent)

A small "front desk" agent decides who should handle the request, then hands it over.

- Flow: User asks, the triage agent picks the right specialist, the specialist uses its tools, then answers.
- Examples:
  - Support desk: triage sends the user to the billing, tech or refund agent
  - Travel planner: separate flight, weather and budget agents
  - Stack: an in-app help agent answers questions and passes bug reports to a bug-triage agent

> READ THE PAPER
> ▸ Handoffs (OpenAI Agents SDK docs): how a triage agent hands a user over to a specialist.
> ▸ How we built our multi-agent research system (Anthropic): a lead agent splitting work across sub-agents.

3. Chat with your documents (RAG)

The agent first finds the right pieces of your documents, then answers from them.

- Flow: User uploads files, the app breaks them into small pieces and stores them. When the user asks, the agent searches those pieces and answers from

what it found.

- Examples:
  - Chat with your PDFs
  - Company handbook assistant
  - "Careful RAG": if the found documents are poor, the agent rewrites the question and searches again

> READ THE PAPER
> ▸ Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020): the original RAG paper: find pieces first, then answer.
> ▸ Self-RAG (Asai et al., 2023): the model judges its own search results before answering.
> ▸ Corrective RAG (Yan et al., 2024): the "careful RAG" idea: poor results, so search again.

4. Structured output (fill a fixed form)

The agent doesn't chat. It returns data in a fixed shape the app can use directly.

- Flow: Input goes in, the agent reads and sorts it, and a clean structured result comes out to be saved or shown.
- Examples:
  - Meeting notes: transcript in, summary and task list out
  - Resume screener: CVs in, a score for each job out
  - Stack "Remember": your highlights in, quiz questions out

> READ THE PAPER
> ▸ Introducing Structured Outputs in the API (OpenAI): forcing the answer into a fixed JSON shape.
> ▸ Efficient Guided Generation for LLMs (Willard & Louf, 2023): the method behind fill-a-fixed-form output.

5. Human approval (pause and ask)

The agent prepares an action but stops and waits for a person to say OK before doing it.

- Flow: User asks, the agent prepares the action, then pauses. The user approves or edits, and only then does the agent carry it out.
- Examples:
  - Email draft: the agent writes it, you approve, then it sends
  - DevOps helper: it suggests a fix and runs it only after you approve
  - Mitra AI: voice order, find items, confirm with the user, place the order, alert family

- Harvey: draft-only cold emails that you check before sending

> READ THE PAPER
> ▸ Human-in-the-loop (LangGraph docs): pause, ask a person, then resume.
> ▸ Building effective agents (Anthropic, 2024): the clearest guide to what an agent is and the common app patterns.

6. Coding / file agent

The agent works on files and runs commands on a computer.

- Flow: You give a task, the agent reads files, runs commands, edits, and checks again until it's done.
- Examples:
  - PR reviewer that reads the changes and writes review comments
  - Docs writer that scans the code and writes the README
  - Bug fixer: run tests, find the cause, edit, re-test
  - Stack: builds the APK, runs emulator tests, and reports what broke and why

> READ THE PAPER
> ▸ SWE-agent: Agent-Computer Interfaces (Yang et al., 2024): an agent that reads code, runs commands, and fixes bugs.
> ▸ SWE-bench (Jimenez et al., 2023): the test used to measure coding agents on real GitHub issues.

7. Team of role agents (crew)

Several agents with job titles work one after another, each passing its work to the next.

- Flow: You give a topic, the researcher gathers facts, the analyst sorts them, and the writer produces the final piece.
- Examples:
  - Blog crew: researcher and writer
  - Market report: researcher, analyst and writer
  - Job apply crew: read the job post, tailor the resume, write the cover letter
  - Stack launch: a marketing crew writes LinkedIn posts, Play Store text and emails

READ THE PAPER

▸ MetaGPT: Meta Programming for Multi-Agent Collaboration (Hong et al., 2023): agents with job titles passing work down a line.

▸ AutoGen: Multi-Agent Conversation (Wu et al., 2023): several agents talking to each other to finish a task.

8. Background / scheduled agent

It runs on its own, on a timer or when an event happens, with no user waiting.

- Flow: The timer fires, the agent runs the loop, saves a report or sends an alert, and the user sees it later.
- Examples:
  - Weekly competitor watch report
  - Daily standup summary from the day's code changes
  - Job leads: reads your leads sheet and drafts one email per company for you to check

READ THE PAPER

▸ Durable execution (LangGraph docs): long jobs that survive restarts and run in the background.

▸ 12-Factor Agents (HumanLayer): rules for building agents that are real, testable software.

9. Voice agent

Same loop, but the user speaks and hears the answer.

- Flow: The user speaks, speech becomes text, the agent loops with its tools, and the answer is spoken back.
- Examples:
  - Voice receptionist that books appointments using a calendar tool
  - Mitra AI voice grocery ordering

READ THE PAPER

▸ Voice agents guide (OpenAI docs): speech in, agent loop, speech out.

▸ Whisper: Robust Speech Recognition (Radford et al., 2022): the speech-to-text step.

10. Mobile app + agent

The phone app has no safe place for secret keys, so it always goes through your server.

- Flow: The user taps a button in the phone app, the request goes to your server, the server checks the user and runs the agent, and the result comes back to the phone. The phone can save it for offline use.

- Example: Stack "Quiz me": the phone asks the server, the agent reads your highlights, and you get a quiz on your phone

> READ THE PAPER
> ▸ OWASP Mobile Top 10: why secret keys must never ship inside a phone app.

11. Local / private agent

The AI model runs on your own computer, so no data leaves it.

- Flow: Same loop, but everything stays on your machine. It usually asks for approval before changing anything.
- Example: a private notes or file assistant running on Ollama

> READ THE PAPER
> ▸ Ollama (GitHub): run a model on your own computer.
> ▸ llama.cpp (GitHub): the engine that makes local models fast.

VI. FIVE RULES EVERY REAL AGENTIC APP FOLLOWS

1. Secret keys stay on the server, never in the website or phone app.
2. The app decides whose data is used. The server passes the logged-in user to the tools, and the agent can never choose another user's data.
3. The agent lives in its own folder, so it's easy to test, improve, or switch models.
4. Answers stream back word by word. Long jobs go to a background worker.
5. Risky actions need human approval: sending, paying, deleting, deploying.

> READ THE PAPER
> ▸ OWASP Top 10 for LLM Applications: the main risks: prompt injection, too much agency, data leaks.
> ▸ Not what you've signed up for: Indirect Prompt Injection (Greshake et al., 2023): why the agent must never pick whose data it reads.

VII. IN ONE LINE

The user asks the app, the server checks who they are and starts the agent, and the agent loops (think, use a tool, read the result) until it has the answer. The answer streams back to the user. Every example above is this same flow with different tools, different numbers of agents, or a pause for approval.

VIII. READING LIST

Start with the first three. The rest go with the section they are linked from.

1. Building effective agents (Anthropic, 2024)
   https://www.anthropic.com/engineering/building-effective-agents
2. LLM Powered Autonomous Agents (Lilian Weng, 2023)
   https://lilianweng.github.io/posts/2023-06-23-agent/
3. A Survey on LLM-based Autonomous Agents (Wang et al., 2023)
   https://arxiv.org/abs/2308.11432
4. ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2022)
   https://arxiv.org/abs/2210.03629
5. Toolformer: Language Models Can Teach Themselves to Use Tools (Schick et al., 2023)
   https://arxiv.org/abs/2302.04761
6. MRKL Systems (Karpas et al., 2022)
   https://arxiv.org/abs/2205.00445
7. Tool use with Claude (Anthropic docs)
   https://docs.anthropic.com/en/docs/build-with-claude/tool-use
8. Streaming messages (Anthropic docs)
   https://docs.anthropic.com/en/docs/build-with-claude/streaming
9. 12-Factor Agents (HumanLayer)
   https://github.com/humanlayer/12-factor-agents
10. A Practical Guide to Building Agents (OpenAI, PDF)
    https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf
11. Writing effective tools for agents (Anthropic)
    https://www.anthropic.com/engineering/writing-tools-for-agents
12. Handoffs (OpenAI Agents SDK docs)
    https://openai.github.io/openai-agents-python/handoffs/
13. How we built our multi-agent research system (Anthropic)
    https://www.anthropic.com/engineering/multi-agent-research-system
14. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)
    https://arxiv.org/abs/2005.11401
15. Self-RAG (Asai et al., 2023)
    https://arxiv.org/abs/2310.11511
16. Corrective RAG (Yan et al., 2024)
    https://arxiv.org/abs/2401.15884

17. Introducing Structured Outputs in the API (OpenAI)
https://openai.com/index/introducing-structured-outputs-in-the-api/

18. Efficient Guided Generation for LLMs (Willard & Louf, 2023)
https://arxiv.org/abs/2307.09702

19. Human-in-the-loop (LangGraph docs)
https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/

20. SWE-agent: Agent-Computer Interfaces (Yang et al., 2024)
https://arxiv.org/abs/2405.15793

21. SWE-bench (Jimenez et al., 2023)
https://arxiv.org/abs/2310.06770

22. MetaGPT: Meta Programming for Multi-Agent Collaboration (Hong et al., 2023)
https://arxiv.org/abs/2308.00352

23. AutoGen: Multi-Agent Conversation (Wu et al., 2023)
https://arxiv.org/abs/2308.08155

24. Durable execution (LangGraph docs)
https://langchain-ai.github.io/langgraph/concepts/durable_execution/

25. Voice agents guide (OpenAI docs)
https://platform.openai.com/docs/guides/voice-agents

26. Whisper: Robust Speech Recognition (Radford et al., 2022)
https://arxiv.org/abs/2212.04356

27. OWASP Mobile Top 10
https://owasp.org/www-project-mobile-top-10/

28. Ollama (GitHub)
https://github.com/ollama/ollama

29. llama.cpp (GitHub)
https://github.com/ggml-org/llama.cpp

30. OWASP Top 10 for LLM Applications
https://genai.owasp.org/llm-top-10/

31. Not what you've signed up for: Indirect Prompt Injection (Greshake et al., 2023)
https://arxiv.org/abs/2302.12173

* * *

NOTES · PART 2

THE LOOP AND THE KNOWLEDGE STEP

Memorandum on Agentic AI Applications · 24 Oct 2026 · continues Part 1

Contents

IX. What Part 2 adds

X. Inside the loop

XI. Inside the knowledge step (RAG and context)

Part 1 showed what an agentic app is made of and pointed each part to a paper. From here on, the notes open those papers and show how each part works inside: the steps, a diagram of the flow, and the few numbers worth knowing. Red underlined titles name the paper or guide; the addresses are in Part 1, section VIII, and the updates in Part 5, section XIX.

IX. WHAT PART 2 ADDS

Part 1 and the 30 paper memos cover the same reading list from two sides. This part joins them.

PART 1 GAVE YOU	PART 2 ADDS	SECTION
The one loop (think, tool, result)	The exact message format and how tools are chosen	X
Chat with your documents	The RAG pipeline, self-checking RAG, and the new topic: context engineering	XI
Fill a fixed form, stream the answer	Why the JSON can't break, and the order of stream events	XII
Five rules, human approval, long jobs	Guardrail layers, the 12 factors, how pause/resume and crash-recovery work	XIII
Router, research team, crew	Handoff vs manager, the real costs, how crews pass documents	XIV
Coding / file agent	How coding agents are tested and the commands they use	XV
Voice agent, local agent	Inside Whisper, the two voice designs, how local models fit	XVI
OWASP and prompt injection links	How the attack works, and the full risk list mapped to your app	XVII

Shapes used in the diagrams: a shaded box is the AI model or agent, a red dashed box is a risk or a guard, plain boxes are people, tools, data and results. A dashed arrow means "go back and repeat".

X. INSIDE THE LOOP

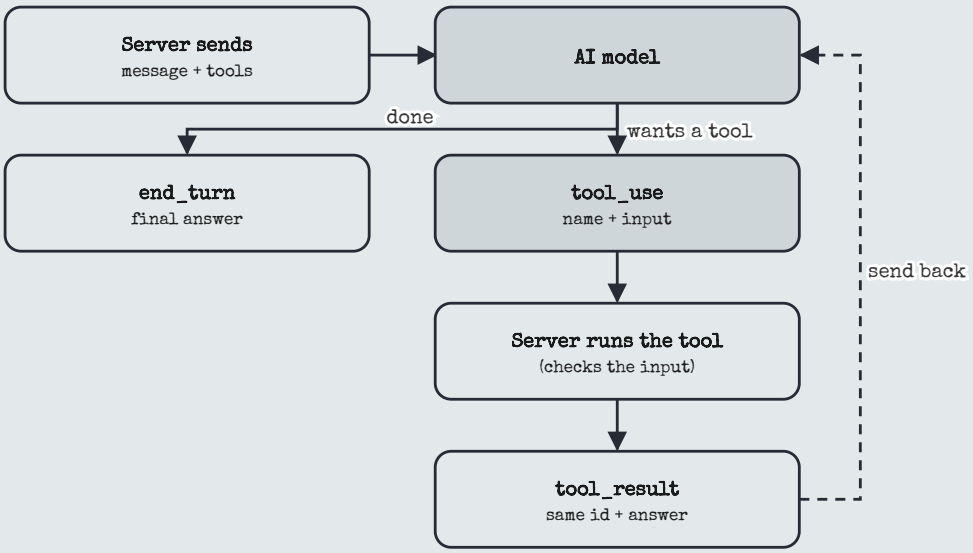
1. The loop as the ReAct paper wrote it

The model writes three kinds of lines, again and again:

```
Thought: I need the order data first.
Action: lookup_order[123]
Observation: Shipped, arrives 22 Oct.
Thought: Now I can answer.
Action: finish[Your order arrives on 22 October.]
```

- Your code stops the model after each *Action*, runs it, and pastes the *Observation* back in.
- Mixing thinking with real results makes up far fewer facts than thinking alone. On two decision tasks (ALFWorld and WebShop) this beat older trained methods by 34 and 10 points.

2. The same loop in real code (tool use)



The server's job in every agent app: send tools, run the tool the model asks for, send the result back.

1. Each tool has a name, a description (3 to 4 clear sentences) and an input shape (JSON Schema).
2. The model answers with a `tool_use` block. It can ask for several tools at once.
3. The server runs the tool and replies with `tool_result` carrying the same id. If it failed, mark it as an error so the model can fix itself.
4. Repeat until the model stops with `end_turn`.

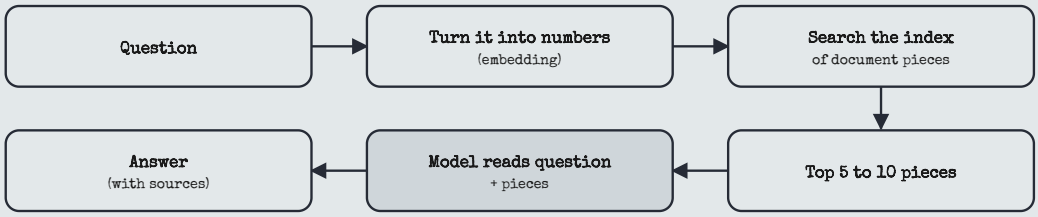
3. How the model picks a tool

- MRKL: a router sends each part of the job to an expert (calculator, database, another model). The hard part is pulling the right inputs out of the user's words, so let code do the exact work.
- Toolformer: the model learned tool calls by itself. It kept a practice call only if the result made the following text easier to predict. A 6.7B model with tools beat the 175B GPT-3 on maths and fact questions.
- Good tools matter more than clever prompts: few tools that don't overlap, clear names (`user_id`, not `user`), short answers, error messages that say how to fix the problem. Test tools with a list of real tasks, then improve them.

> READ THE PAPER
>
> ▸ ReAct (Yao et al., 2022): the Thought, Action, Observation loop
>
> ▸ Tool use with Claude (Anthropic docs): tool_use and tool_result in code
>
> ▸ Writing effective tools for agents (Anthropic): design tools, then test them with evals

XI. INSIDE THE KNOWLEDGE STEP (RAG AND CONTEXT)

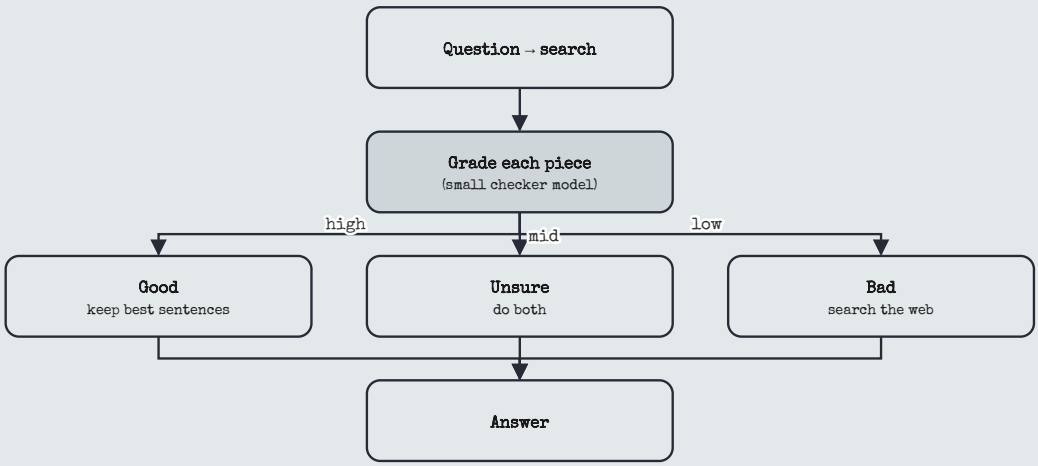
1. The original RAG pipeline



Retrieval-augmented generation: find pieces first, then answer from them.

- The original paper split Wikipedia into 21 million pieces of 100 words, searched them, and let a small model answer from the top 5 to 10.
- Swapping the index for a newer one updated the model's knowledge without retraining. This is why your data stays fresh.

2. Careful RAG: two ways to check the search



Corrective RAG: grade the search results, then choose what to do.

- Self-RAG: the model writes small check-marks into its own answer: *Search?* (yes/no), *Relevant?*, *Supported?*, *Useful?* (1 to 5). It searches only when needed and keeps the best-supported answer.
- Corrective RAG: a small grader scores the results. Good means trim them to the useful sentences. Bad means throw them away and search the web. Unsure means do both.
- You can copy both with prompts: ask "do I need to search?" first and "is this answer backed by the sources?" last.

3. New topic: context engineering

Part 1 didn't cover this. The context window is the agent's working memory, and it has a budget. The more you put in, the worse the model gets at using it. So choose, at every turn, the smallest set of useful words.

PROBLEM	WHAT TO DO	EXAMPLE
Instructions too strict or too vague	Write them at the "right height": clear sections, only rules that fix real mistakes	System prompt with headings
Too much data loaded up front	Load it when needed: keep file names or links, fetch with a tool	Search files, then open one
Long chat fills the window	Compaction: summarise, keep decisions and open problems, drop old tool results	Long coding session
Agent forgets across sessions	Notes outside the window that it reads back	NOTES.md, to-do list
One job needs lots of reading	Sub-agents read in their own window and return a short summary	Research helpers

> READ THE PAPER
>
> - Retrieval-Augmented Generation (Lewis et al., 2020): the original pipeline
> - Self-RAG (Asai et al., 2023): search only when needed, check support
> - Corrective RAG (Yan et al., 2024): grade results, search the web if bad
> - Effective context engineering for AI agents (Anthropic): the new topic in this part

* * *

Continued in Part 3.

NOTES · PART 3

CLEAN OUTPUT AND PRODUCTION AGENTS

Memorandum on Agentic AI Applications · 24 Oct 2026 · continues Part 2

Contents

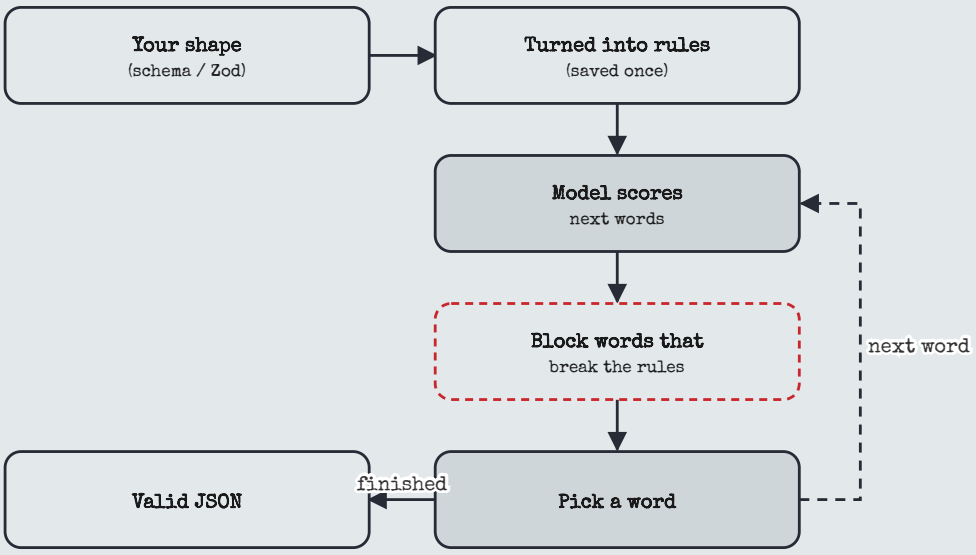
XII. Inside clean output and streaming

XIII. Inside a production agent

Diagram key: a shaded box is the AI model or agent, a red dashed box is a risk or a guard, plain boxes are people, tools, data and results. A dashed arrow means "go back and repeat".

XII. INSIDE CLEAN OUTPUT AND STREAMING

1. Why the fixed-form answer cannot break



Structured output: words that would break the shape are blocked before the model can pick them.

1. Your shape is turned into a set of rules (a state machine or grammar). This happens once and is saved, so the first call is a little slower.
2. At every word, the rules list which words are allowed next. All others get a chance of zero.
3. The model picks only from the allowed words, so the output always fits the shape.
4. Result from OpenAI's launch: 100% on their hard shape test, against under 40% for an older model without it.

- The shape is guaranteed. The values are not. Keep checking them in code (prices, totals, IDs).

2. The order of streaming events

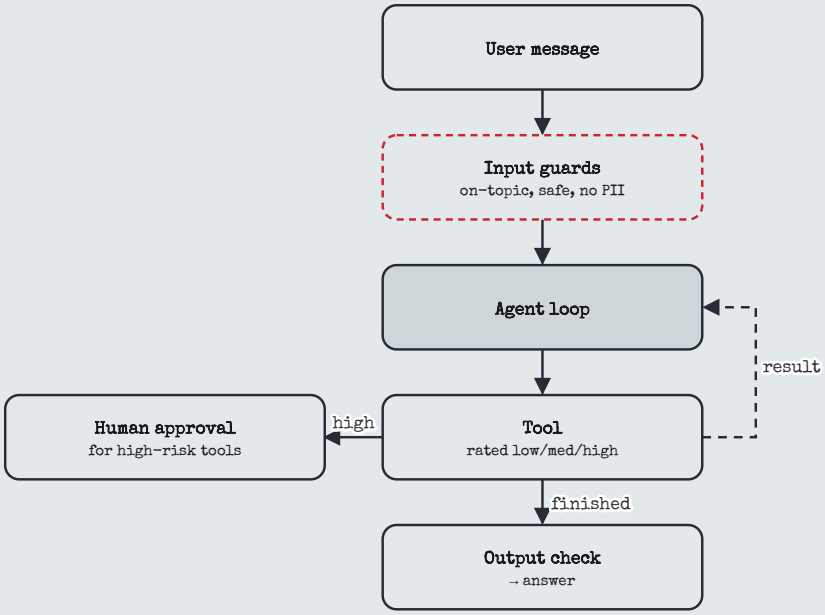
EVENT	WHAT IT CARRIES
message_start	an empty message
content_block_start	a new block begins (text or tool call)
content_block_delta	a small piece: text, part of a tool's JSON, or thinking
content_block_stop	that block is done
message_delta	why it stopped, and the token count
message_stop	all done (errors can also arrive mid-stream)

- Show text pieces as they arrive. Only act (save, pay, send) on the finished message.

> READ THE PAPER
>
> ▸ Introducing Structured Outputs (OpenAI): strict mode and the 100% result
>
> ▸ Efficient Guided Generation (Willard & Louf, 2023): the blocking method inside
>
> ▸ Streaming messages (Anthropic docs): every event type

XIII. INSIDE A PRODUCTION AGENT

1. Guards in layers



No single guard is enough, so stack them.

- The loop ends when the model gives a final answer, calls no tool, hits an error, or reaches the turn limit.
- Start with one agent. Split into several only when the instructions get too tangled or the tools overlap.

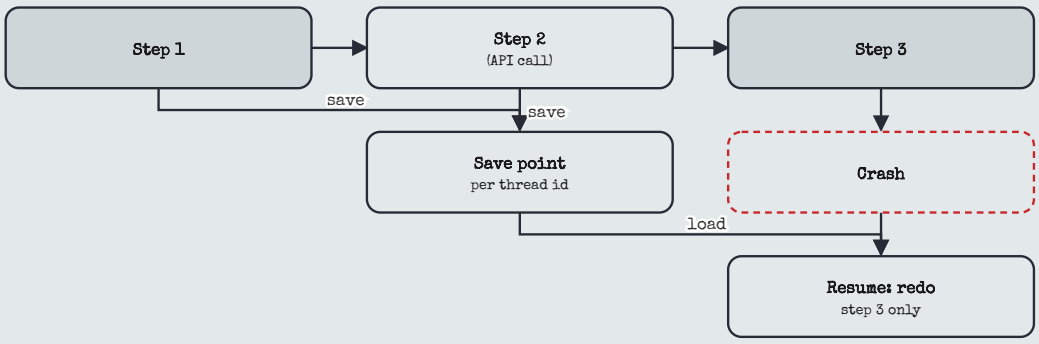
2. The 12 factors on one page

#	FACTOR	IN PLAIN WORDS
1	Natural language to tool calls	The model turns words into a structured action
2	Own your prompts	No hidden prompts from a framework
3	Own your context window	You decide what goes in, in what format
4	Tools are structured outputs	A tool call is just JSON your code acts on
5	One state	Agent progress and business data live together
6	Start, pause, resume	Simple calls to control a run
7	Contact humans with tools	"Ask for approval" is just another tool
8	Own your control flow	Your code decides what happens next
9	Compact errors	Put short error notes back in, with a retry limit
10	Small, focused agents	3 to 10 steps each, not 100
11	Trigger from anywhere	Slack, email, timer, webhook
12	Stateless reducer	(state, event) → new state

3. How pause-and-ask really works

1. The agent reaches a risky step and calls `interrupt(...)` with what it wants to do.
2. Everything is saved under a thread id (a save slot), and the run stops.
3. Your app shows the request. The person approves, edits or rejects.
4. You resume the same thread id with their answer.
5. Watch out: the paused step starts again from its top, so anything before the pause must be safe to run twice.

4. How long jobs survive a crash



Save after every step; after a crash only the unfinished step runs again.

- Save modes: at the end only (fast, risky), in the background, or before every next step (safest).
- Put side effects (emails, payments) in their own saved step and give them an idempotency key, so a retry never sends twice.

> READ THE GUIDE
>
> ▸ A Practical Guide to Building Agents (OpenAI, PDF): guards and when to split agents
>
> ▸ 12-Factor Agents (HumanLayer): the 12 rules
>
> ▸ Human-in-the-loop (LangGraph docs): interrupt and resume
>
> ▸ Durable execution (LangGraph docs): save points and save modes

* * *

Continued in Part 4.

NOTES · PART 4

TEAMS, CODING, VOICE AND LOCAL AGENTS

Memorandum on Agentic AI Applications · 24 Oct 2026 · continues Part 3

Contents

XIV. Inside a team of agents

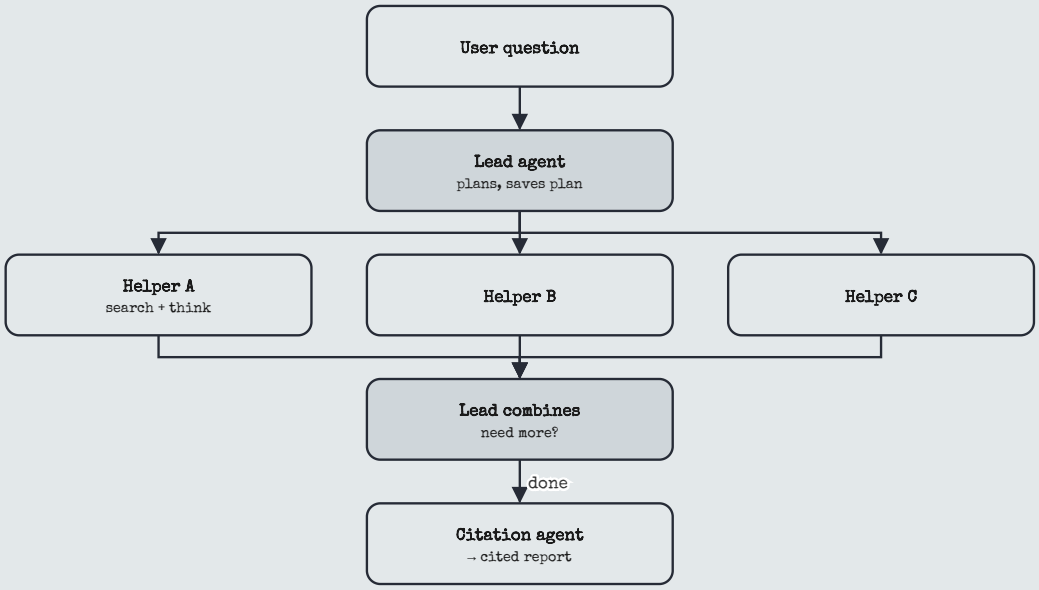
XV. Inside coding agents

XVI. Inside voice and local agents

Diagram key: a shaded box is the AI model or agent, a red dashed box is a risk or a guard, plain boxes are people, tools, data and results. A dashed arrow means "go back and repeat".

XIV. INSIDE A TEAM OF AGENTS

PATTERN	WHO IS IN CHARGE	USE IT FOR
Handoff	Control moves to the specialist, who talks to the user	Support desk: billing, tech, refunds
Manager (agents as tools)	The main agent keeps control, borrows answers	One voice to the user, many helpers
Lead + parallel helpers	A lead plans, helpers search at the same time	Research, comparisons
Conversation (AutoGen)	Agents message each other until done	Write code, run it, fix it
Crew with procedures (MetaGPT)	Each role writes a document for the next	Small software projects



Lead and parallel helpers, as in Anthropic's Research feature.

- A handoff is just a tool named like `transfer_to_billing`. Calling it swaps which agent is in charge.
- Anthropic's team: lead plus helpers beat a single top model by 90% on their research test, but used about 15 times the tokens of a normal chat. Worth it only for valuable, splittable work.
- How much effort to spend: a simple fact needs 1 agent and 3 to 10 tool calls. A comparison needs 2 to 4 helpers. A big topic needs 10 or more.
- Crews work better when roles pass documents (plan, design, task list) instead of chatting. MetaGPT reached 85.9% on a standard coding test this way.
- The 2023 AutoGen code is out of date. Microsoft rebuilt it and now points to Microsoft Agent Framework.

READ THE PAPER

- Handoffs (OpenAI Agents SDK docs): transfer tools
- How we built our multi-agent research system (Anthropic): the real costs and lessons
- AutoGen (Wu et al., 2023): agents that talk
- MetaGPT (Hong et al., 2023): roles and documents

XV. INSIDE CODING AGENTS

1. How coding agents are tested (SWE-bench)

1. Take real fixed bugs from 12 popular Python projects: 2,294 tasks.
2. Give the agent the bug report and the code from before the fix.
3. The agent writes a patch.
4. Run the project's tests: the ones that failed must now pass, and the ones that passed must still pass.
5. At launch the best model fixed under 2%. A cleaned-up set of 500 human-checked tasks ("Verified") is now the usual scoreboard.

2. The commands a good coding agent gets (SWE-agent)

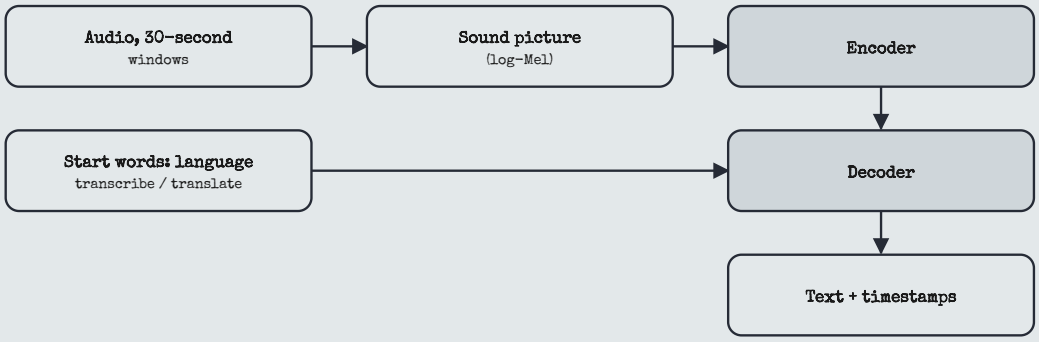
COMMAND	WHY IT HELPS
find_file, search_dir	Short summary of matches (max 50), not a flood
open, goto, scroll	Shows 100 numbered lines at a time
edit start:end	Replaces lines; a syntax checker rejects broken edits
(history)	Old outputs are folded to one line to save space
submit	Turns the changes into a patch

- With these commands, GPT-4 Turbo fixed 12.5% of the full test, against 3.8% before. Today's agents score far higher, but the lesson stays: design the agent's commands like a good UI.

> READ THE PAPER
>
> ▸ SWE-bench (Jimenez et al., 2023): the test
>
> ▸ SWE-agent (Yang et al., 2024): the commands

XVI. INSIDE VOICE AND LOCAL AGENTS

1. Inside the speech-to-text step (Whisper)



Whisper: one model for many speech jobs, picked by special start words.

- Trained on 680,000 hours of real internet audio, so it copes with accents and noise. It made about 55% fewer errors than a model trained on clean audio.
- Weak spots: it can invent words during silence and repeat itself on long audio. Cut out silence first.

2. The two ways to build a voice agent

	CHAINED	SPEECH-TO-SPEECH
Flow	speech-to-text → text agent → text-to-speech	one realtime model, audio in and out
Good at	control, transcripts, reuse your text agent	speed, natural voice, hears tone
Watch out	delay adds up at each stage, so stream each one	less control over exact words, costs more
Start here if	you need Hindi support, logs, strict rules	speed matters most

3. How a local model fits on your laptop

SIZE OF MODEL	4-BIT (Q4) MEMORY, ROUGHLY	8-BIT (Q8) MEMORY, ROUGHLY
3B	about 2 GB	about 3.5 GB
7 to 8B	about 5 GB	about 8.5 GB
13 to 14B	about 8.5 GB	about 15 GB

- Rule of thumb: about 0.6 GB per billion parameters at 4-bit, plus extra for long chats.
- llama.cpp is the engine (GGUF files, CPU or GPU). Ollama wraps it with easy commands and a local API at `localhost:11434` that works with OpenAI-style code.
- Good split: a local model for cheap steps (sorting, pulling out fields), a cloud model for hard steps.

READ THE PAPER

- Whisper (Radford et al., 2022): inside speech-to-text
- Voice agents guide (OpenAI docs): chained vs speech-to-speech
- Ollama (GitHub): easy local models
- llama.cpp (GitHub): the engine

* * *

Continued in Part 5.

NOTES · PART 5

SECURITY, NUMBERS AND LINK UPDATES

Memorandum on Agentic AI Applications · 24 Oct 2026 · continues Part 4

Contents

XVII. Inside the security rules

XVIII. Numbers worth remembering

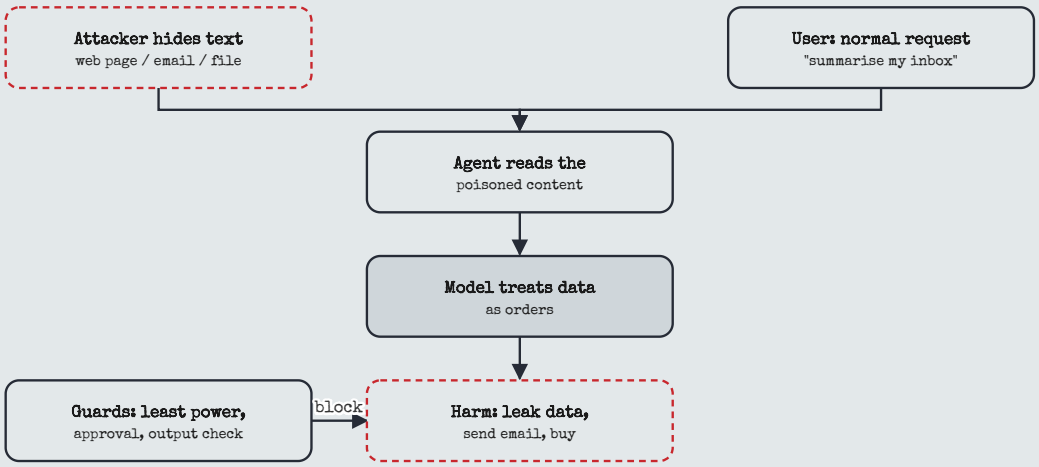
XIX. Link updates for Part 1

XX. In one line

Diagram key: a shaded box is the AI model or agent, a red dashed box is a risk or a guard, plain boxes are people, tools, data and results. A dashed arrow means "go back and repeat".

XVII. INSIDE THE SECURITY RULES

1. How indirect prompt injection works



The attack arrives through data the agent reads, not from the user.

- Hidden text can be white-on-white, in HTML comments, or fetched in a second step. One poisoned web page turned Bing Chat into a scammer against its own user.
- There is no complete fix yet. Limit what the agent is allowed to do, ask a human before risky actions, and never let the agent choose whose data it reads (Part 1, rule 2).

2. The OWASP list mapped to your app

RISK (LLM TOP 10, 2025)	WHERE IT HITS	PART 1 RULE THAT HELPS
01 Prompt injection	anything the agent reads	2, 5
02 Sensitive info leaks	answers and logs	2
03 Supply chain	models, packages, plugins	3
04 Data and model poisoning	documents in your RAG	3
05 Unchecked output	output passed to code or HTML	check before use
06 Too much agency	tools that act	5
07 System prompt leaks	instructions	1 (keep secrets out of prompts)
08 Vector store weaknesses	RAG index	2
09 Misinformation	answers	RAG checks (XI)
10 Unbounded use (cost)	loops, big inputs	turn limits, spend caps

- For phone apps, the Mobile Top 10 (2024) starts with improper credential use: the reason Part 1 says keys never ship in the app.
- Some news sites reported a 2026 edition of the LLM list in August 2026. The official page still showed 2025 on 24 Oct 2026, so check before quoting it.

> READ THE PAPER
>
> ▸ Indirect Prompt Injection (Greshake et al., 2023): the attack in detail
>
> ▸ OWASP Top 10 for LLM Applications 2025: the full list
>
> ▸ OWASP Mobile Top 10: phone app risks

XVIII. NUMBERS WORTH REMEMBERING

NUMBER	WHAT IT MEANS	FROM
+34 / +10 pts	ReAct over older trained methods on two decision tasks	ReAct
6.7B beats 175B	A small model with tools beat GPT-3 on maths and fact questions	Toolformer
21 million	Wikipedia pieces searched in the first RAG paper	RAG
100% vs under 40%	Shape-following with strict structured output vs an older model	Structured Outputs
90% better, 15× tokens	Lead + helpers vs one agent, and the cost	Anthropic research system
85.9%	MetaGPT on a standard coding test (2023)	MetaGPT
2,294 tasks / under 2%	SWE-bench size, and the best score at launch	SWE-bench
12.5% vs 3.8%	Better agent commands vs plain search	SWE-agent
680,000 hours	Audio used to train Whisper	Whisper
about 0.6 GB per 1B	Memory for a 4-bit local model	llama.cpp / Ollama

XIX. LINK UPDATES FOR PART 1

Some links in Part 1's reading list now redirect. They still work, but these are the current addresses:

PART 1 NO.	TITLE	CURRENT LINK
1	Building effective agents	https://www.anthropic.com/research/building-effective-agents
7	Tool use with Claude	https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview
8	Streaming messages	https://platform.claude.com/docs/en/build-with-claude/streaming
11	Writing effective tools for agents	https://www.anthropic.com/research/writing-tools-for-agents
13	Multi-agent research system	https://www.anthropic.com/research/multi-agent-research-system
19	Human-in-the-loop (LangGraph)	https://docs.langchain.com/oss/python/langgraph/interrupts
24	Durable execution (LangGraph)	https://docs.langchain.com/oss/python/langgraph/durable-execution
25	Voice agents guide	https://developers.openai.com/api/docs/guides/voice-agents
new	Effective context engineering	https://www.anthropic.com/research/effective-context-engineering-for-ai-agents

For a full memo on each paper, see *Agentic AI Paper Memos.pdf*. For videos, see *Agentic AI YouTube videos.pdf*. Both are on the Desktop.

XX. IN ONE LINE

Inside every part is the same idea: let the model decide, but let code check. Tools return short, exact answers. Search results are graded. Output shapes are enforced. Risky steps wait for a human. Progress is saved after every step. Everything the agent reads is treated as data, never as orders.

* * *